README

HelixCode Deployment Guide — Zero-Bluff Phase 5

Audience: Operators deploying HelixCode in production. **Companion**: Comprehensive references at `docs/COMPLETE_DEPLOYMENT_GUIDE.md` and `docs/DEPLOYMENT_GUIDE.md`. **Mandate**: CONST-033 (no host power management), CONST-042 (no secret leak), CONST-043 (no force push), CONST-045 (no hardcoded distribution hosts).
Last updated: 2026-05-12

---

1. containers Submodule

HelixCode ships an orchestration layer in the `containers/` submodule. All container artefacts live there — `Dockerfiles`, `compose`/`podman-compose` files, environment templates.

```
git submodule update --init Containers
cd Containers
./bin/orchestrator up       # bring up the platform stack
./bin/orchestrator down
./bin/orchestrator status
```

The `./helix` facade in the repo root delegates to the orchestrator:

```
./helix start
./helix stop
./helix logs
./helix shell
```

Rule 4 (Constitution): never invoke `docker` / `docker-compose` directly. The orchestrator binary is the only supported workflow.

---

2. Configuration

2.1 Env-var precedence

1. CLI flag
2. Process environment
3. `~/.config/helixcode/config.yaml`
4. Repo-local `.helixcode.yaml`
5. Built-in defaults

2.2 API keys

API keys live ONLY in `~/.config/helixcode/api_keys.sh` (mode 0600). Load:

```
source ~/.config/helixcode/api_keys.sh
./bin/cli
```

Per CONST-042, secrets MUST NOT appear in any repository, image, or log.

2.3 Distribution hosts (CONST-045)

Container distribution targets are configured **exclusively** through `CONTAINERS_REMOTE_HOST_N_*` env vars in `containers/.env` (mode 0600, gitignored).

Adding / removing a host is an edit to `containers/.env` — never a code change. Tests read `.env` at runtime and skip with `SKIP-OK:` when `CONTAINERS_REMOTE_ENABLED=false`.

2.4 Reference `config.yaml`

```yaml
approval:
  mode: auto-edit
  config_file: ~/.config/helixcode/approval.yaml
sandbox:
  profile: workspace-write
providers:
  default: anthropic
  anthropic:
    model: claude-3-5-sonnet
session:
  retention_days: 30
memory:
  path: .helixcode/memory.md
telemetry:
  enabled: true
  endpoint: http://localhost:4317
  service_name: helixcode
quality:
  gate: strict
```

Full field reference: docs/COMPLETE_CONFIGURATION_DOCUMENTATION.md.

---

3. Production Hardening

3.1 Mandatory settings

Setting	Production value	Rationale
`approval.mode`	`auto-edit` (interactive) or `full-auto` (CI-style)	Never `dangerously-bypass`
`sandbox.profile`	`workspace-write` minimum	`full-auto` REFUSES without sandbox

Setting	Production value	Rationale
`telemetry.enabled`	`true`	Visibility into approval/ sandbox denials
`quality.gate`	`strict`	Reject low-confidence LLM outputs
File mode for `api_keys.sh`	`0600`	CONST-042
File mode for `containers/.env`	`0600`	CONST-045

3.2 Approval matrix in production

Mode	Use case	Required guardrails
`suggest`	Pair-programming session	Operator at keyboard
`auto-edit`	Background agent edits	Sandbox + telemetry
`full-auto`	Unattended CI-style runs	Sandbox MANDATORY, broker-mediated approval ack
`dangerously-bypass`	Forbidden in prod	—

The executor enforces `ErrSandboxRequired` when `full-auto` + no sandbox.

3.3 Logging

`internal/logging` writes JSON-lines logs to stderr by default. Redirect to a log shipper (e.g. Vector, Fluent Bit). Never log raw memory file contents (CONST-042); the loader logs only path + byte counts.

---

4. Observability

4.1 OpenTelemetry endpoint

```
HELIXCODE_OTEL_ENDPOINT=http://otel-collector:4317 \
HELIXCODE_OTEL_SERVICE_NAME=helixcode-prod \
./bin/cli
```

Default transport: gRPC. Use `HELIXCODE_OTEL_PROTOCOL=http/protobuf` for HTTP.

4.2 Metrics catalogue

- `helixcode.llm.tokens_in` / `tokens_out` (counter, attrs: provider, model)
- `helixcode.llm.request_duration_ms` (histogram)
- `helixcode.approval.denials_total` (counter, attrs: mode, level, reason)
- `helixcode.sandbox.violations_total` (counter)
- `helixcode.tools.execute_duration_ms` (histogram, attrs: tool_name)
- `helixcode.session.active_sessions` (gauge)
- `helixcode.task.checkpoints_total` (counter)

4.3 Dashboards

Reference dashboard at `containers/dashboards/grafana-helixcode.json` (when present). Key panels:

- Approval denials by mode/level (24h)
- Sandbox violation rate (1h sliding)
- LLM token consumption per provider
- p50 / p95 / p99 request latency per model

4.4 Slash command for runtime inspection

```
/telemetry status        # current span ID, exporter health, queue depth
/telemetry flush         # force flush before shutdown
```

---

5. CI/CD — Manual Workflow Only

Rule 1 (Constitution): no CI/CD pipelines. No `.github/workflows/`, no `.gitlab-ci.yml`, no Jenkinsfile. All builds and tests run manually or via `Makefile` / script targets.

5.1 Build target chain

```
# 1. Compile-only smoke
cd HelixCode && make verify-compile

# 2. Unit tests (mocks allowed only here)
cd HelixCode && go test -count=1 ./...

# 3. Anti-bluff scan
grep -rn "simulated\|for now\|TODO implement\|placeholder" \
  helix_code/internal helix_code/cmd && echo "BLUFF FOUND" ||
        echo "clean"

# 4. Real LLM end-to-end (requires `make test-infra-up` first)
make test-infra-up
make test-full
make test-infra-down

# 5. Governance cascade
./scripts/verify-governance-cascade.sh
```

5.2 Release build

```
cd HelixCode && make prod
# → bin/helixcode-linux-amd64
#   bin/helixcode-darwin-amd64
#   bin/helixcode-darwin-arm64
#   bin/helixcode-windows-amd64.exe
```

For containerised builds without host Go:

```
make container-builder-image    # once
make container-build
make container-test
make container-release
```

5.3 Manual deployment cadence

1. `git pull --ff-only` on every remote (origin / github / gitlab / upstream).
2. `make test-full` against staging stack.
3. Tag release: `git tag -s v<x.y.z> -m "..."`.
4. Push to four remotes per CONST-043:

   ```
   for R in origin github gitlab upstream; do git push "$R"
           main; git push "$R" --tags; done
   ```

5. `./helix stop && ./helix start` on each host.

---

6. Backup & Restore

6.1 Session transcripts (F11)

Location: `~/.local/share/helixcode/sessions/<id>/`. Each session contains `transcript.jsonl`, `tool_results.jsonl`, `metadata.json`.

Backup:

```
tar czf helixcode-sessions-$(date +%Y%m%d).tar.gz \
  ~/.local/share/helixcode/sessions/
```

Restore: untar into the same location; `cli sessions list` immediately picks up restored sessions.

6.2 Project memory (F24)

Project-root files (`helixcode.md` / `codex.md` / `AGENTS.md`) live in the repo and follow git's backup semantics.

User overlay at `~/.config/helixcode/memory.md` should be backed up separately (mode 0644, no secrets per CONST-042).

6.3 Database (multi-user mode)

PostgreSQL backup (per cluster pg conventions):

```
pg_dump -Fc -d helixcode > helixcode-$(date +%Y%m%d).dump
```

Restore: `pg_restore -d helixcode helixcode-<date>.dump`. Schemas live in `helix_code/internal/database/migrations/`.

---

7. Security

7.1 Secret rotation

API keys in `api_keys.sh`:

1. Generate new key with provider.
2. Edit `api_keys.sh` in place (mode stays 0600).
3. `kill -HUP $(pgrep helixcode)` — process re-reads on signal.
4. Revoke old key with provider.

7.2 Governance cascade verification

Every owned-by-us repo MUST carry `CONSTITUTION.md`, `CLAUDE.md`, `AGENTS.md` with the anti-bluff anchor.

```
./scripts/verify-governance-cascade.sh
# Phase 4 close-out verified 39/39 governance files.
```

7.3 Anti-bluff smoke (always run before release)

```
grep -rn "simulated\|for now\|TODO implement\|placeholder" \
  helix_code/internal helix_code/cmd && echo "BLUFF FOUND" ||
        echo "clean"
```

A `BLUFF FOUND` is a release blocker per Article XI §11.9.

7.4 Host power management

CONST-033: no suspend, hibernate, poweroff, reboot, halt, or any power-state transition in any script, image, systemd unit, or test. Hard ban.

---

8. Troubleshooting Production

Symptom	Diagnosis	Fix
`approval: full-auto mode requires sandbox`	`--no-sandbox` while mode is `full-auto`	Remove flag or change mode
`secret_filter: matched AKIA pattern`	LLM-generated commit subject contained a secret	Subject was scrubbed; investigate source diff
`verifier: model not in catalogue`	Provider/model not registered in LLMsVerifier	Add via verifier CLI; never hardcode
	Collector down	

Symptom	Diagnosis	Fix
`otel: exporter failed: connection refused`		Restart collector; `/telemetry flush`
`task: deps not satisfied`	Dependency chain has a failed task	`cli task show <id>` to inspect chain
`git_auto_commit: skipped (clean tree)`	No mutations to commit	Expected no-op

---

9. Submodule Map (Operator View)

Submodule	Purpose	Required in prod?
`helix_code/`	Core Go application	yes
`helix_qa/`	QA + challenge orchestration	recommended
`challenges/`	Challenge bank	recommended for verification
`containers/`	Docker/container artefacts	yes
`security/`	Security tooling	yes
`assets/`	Logos, themes, brand	optional
`github_pages_website/`	Marketing site	optional
`dependencies/`	LLama_CPP, Ollama, HuggingFace_Hub	as needed

Update all submodules deepest-first when releasing; never force-push (CONST-043).

Sources verified

Sources verified 2026-05-29: https://go.dev/doc/devel/release (Go 1.26 GA, latest patch go1.26.3 — matches project module authority CLAUDE.md §3.1), https://endoflife.date/postgresql (PostgreSQL 15 supported through 2027-11 — confirms the backup/restore guidance targets a still-supported major), https://endoflife.date/redis (Redis 7.4 still receives maintenance patches). Confirmed: OpenTelemetry OTLP default transport is gRPC with `http/protobuf` as the HTTP alternative (matches `HELIXCODE_OTEL_PROTOCOL` guidance); Rule-4 orchestrator-only container workflow (no direct `docker`/`docker-compose`) matches CLAUDE.md §3.4. Negative finding: OTEL endpoints shown (`http://localhost:4317`, `http://otel-collector:4317`) are operator-local / in-cluster addresses, not public service endpoints — nothing external to verify against.